Scientific Forgery Image Detection

By: Janelle Rivera, Eric Marroquin, Alexis Flores, Alex Lam

May 8, 2026

Software Engineering Tools, CS 3338

Table of Contents

1. Introduction

This document describes the design of the Scientific Forgery Image Detection System. The system is a web-based application that allows users to upload biomedical and scientific images and receive analysis results indicating possible copy-move forgery.

The purpose of this design specification is to define how each part of the system is structured, how users interact with it, and how data flows between components.

2. System Overview

The system follows a client-server architecture consisting of:

- **Frontend:** User interface built for uploading and viewing images
- **Backend:** API server that handles image uploads and responses
- **Detection Module:** Image analysis system for identifying forged regions (Planned)

High-Level Goal:

Allow users to upload a scientific image and receive a result indicating whether duplicated regions are detected.

3. User Interface Design

3.1 Home Page

Purpose:

Introduce the system and guide users to the upload feature.

Layout:

- Project title: “Scientific Forgery Image Detection System”
- Short description of purpose
- Navigation button to upload page

3.2 Image Upload Page (implemented, snapshot 2)

Purpose:

Allow users to upload images for analysis.

Components:

- File input Image preview section
- “Analyze Image” button
- Loading indicator

Behavior:

- User selects an image file
- Image is displayed in preview section
- User submits image for processing

3.3 Results Display Section

Purpose:

Show detection output from the backend system.

Components:

- Original uploaded image display

- Result image (highlighted regions)
- Text output message:
- “No forgery detected” OR
- “Possible copy-move forgery detected”

Behavior:

- Displays response after backend processing is complete

4. System Components

4.1 Frontend Component (Partially implemented, snapshot 2)

Responsibilities:

- Provide user interface
- Handle image uploads
- Send data to backend
- Display results

Key Functions:

- Image selection
- Image preview rendering
- API request handling
- UI updates based on response

4.2 Backend Component

Responsibilities:

- Receive image uploads
- Process HTTP requests
- Return response to frontend

Key Functions:

- File upload handling
- API endpoint management
- Response formatting (JSON)

Dependencies: Fastapi, uvicorn, python-multipart

4.3 Detection Module (Planned)

Responsibilities:

- Analyze uploaded images
- Detect duplicated regions
- Generate output masks or annotations

4.4 Database (Planned)

Responsibilities:

- Store uploaded image metadata
- Store detection results
- Maintain upload history

Panned Tech: PostgreSQL

5. Data Flow Design

The system follows this flow:

User uploads image
↓
Frontend (React UI): sends image to backend
↓
Backend (FastAPI server): receives, then validates and processes the image
↓
Detection Module (future): Backend forwards image to detection module
↓
Detection module returns a highlighted image highlighting forged regions
↓
Backend response: formats and returns JSON response to frontend
↓
Frontend displays result

Data Type:

- Image file (JPEG, PNG)
- JSON response from backend
- Segmentation mask (future): from detection module

6. System Behavior

6.1 Image Upload Process

1. User selects image file
2. Frontend stores file locally
3. User clicks “Analyze Image”
4. File is sent to backend API

6.2 Backend Processing

1. Backend receives image
2. Validates file format
3. (Future) Sends image to detection module
4. Returns response to frontend

6.3 Result Display

1. Frontend receives response
2. Displays result message
3. Shows processed image (future feature)

7. Technology Stack

7.1 Frontend

- React.js
- HTML/CSS
- JavaScript

7.2 Backend

- Python
- FastAPI
- Uvicorn

7.3 Planned Libraries

- OpenCV (image processing)
- NumPy (numerical operations)
- PyTorch (machine learning)
- Pillow (image handling)

7.4 DevOps:

- Docker
- Docker Compose

8. Future Enhancements

Future versions of the system may include:

- Deep learning-based forgery detection
- Pixel-level segmentation of manipulated regions
- Heatmap visualization of forged areas
- Improved accuracy models
- Batch image processing
- User authentication system

9. Glossary

Term	Definition
API	System that allows communication between frontend and backend
Backend	Server-side application logic
Frontend	User interface of the system

Copy-Move Forgery	Image manipulation where regions are duplicated within the same image
Detection Module	System responsible for analyzing images
JSON	Data format used for API communication
React.js	JavaScript library for building user interfaces
FastAPI	Python framework for building APIs

10. Version Table

Version	Date	Description	Authors
1.0	May 8, 2026	Initial Design Specification created	Janelle Rivera, Eric Marroquin, Alexis Flores, Alex Lam
1.1	May 10, 2026	Updated to reflect frontend implementation in Snapshot 2: added UI component details and system behavior	Janelle Rivera, Eric Marroquin, Alexis Flores, Alex Lam